Revisión 1.1

Universidad Nacional de Río Negro

Laboratorio de Informática aplicada

Apunte curso “Introducción a Docker”

Unidad 1

Docentes:

- Lic. Horacio Munoz Abbate
  - (hmunoz@unrn.edu.ar)

Ejecutando nuestros primeros comandos Docker **2**

Mi primer Docker **3**
Docker run 3
ref: https://docs.docker.com/engine/reference/commandline/run/ 3
Portainer (Dashboard para Docker) 4
Mysql Local 5

Comando básicos **5**

Dockerizing una aplicación **6**
Dockerfile 6
Dockerfile para aplicación Java Spring Boot 6
Dockerfile multistage 8

Operaciones con la Imagen **9**

Docker Compose Ej: BackEnd Java + MongoDB **9**

Bibliografía **12**

Docker

Ejecutando nuestros primeros comandos Docker

Intentemos iniciar un contenedor iniciando un servidor Ubuntu en uno usando el comando de ejecución de Docker

```
docker run -it --rm ubuntu
```

Con el comando anterior, le pedimos a **Docker** que cree un contenedor que ejecute **Ubuntu**, basado en la última versión disponible de la imagen oficial de Docker para Ubuntu. La opción **-it** se usa para que podamos interactuar con el contenedor usando Terminal, y la opción **--rm** le dice a Docker que elimine el contenedor una vez que salgamos de la sesión de Terminal; de lo contrario, el contenedor permanecerá en el motor Docker con un estado Salido.

La primera vez que usamos una imagen de Docker que no hemos construido, Docker la descarga de un Docker Registry, que es Docker Hub de forma predeterminada (https://hub.docker.com). Esto llevará algún tiempo, pero para el uso posterior de esa imagen de Docker, ¡el contenedor se iniciará en solo unos segundos!

Una vez que se ha descargado la imagen de Docker y se ha iniciado el contenedor, el servidor Ubuntu debe responder con un mensaje como el siguiente:

```
root@6883b2a8ba71: /
(⎈ |minikube:default)➜  ~ docker run -it --rm ubuntu
root@6883b2a8ba71:/#
```

```
cat /etc/os-release | grep 'VERSION='
```

```
root@6883b2a8ba71: /
root@6883b2a8ba71:/# cat /etc/os-release | grep 'VERSION='
VERSION="20.04 LTS (Focal Fossa)"
root@6883b2a8ba71:/#
```

Podemos dejar el contenedor con un comando de salida y verificar que el contenedor de Ubuntu ya **no** salga con el comando **docker ps -a**. Necesitamos usar la opción -a para ver los contenedores detenidos; de lo contrario, solo se muestran los contenedores en ejecución.

Mi primer Docker

Docker run

ref: https://docs.docker.com/engine/reference/commandline/run/

ref: https://docs.docker.com/config/containers/resource_constraints/

- --detach, -d: Ejecutar contenedor en segundo plano e imprimir ID de contenedor

- --name: Asigna un nombre al contenedor
- --expose: Exponer un puerto o un rango de puertos
- --publish, -p: Publica los puertos de un contenedor en el host
- --restart: [no, always,on-failure, unless-stopped] política de reinicio
- --volume, -v, --mount: Bind monta un volumen
- --link: Agregar enlace a otro contenedor
- --env, -e Establecer variables de entorno

- **Ngnix**:

https://hub.docker.com/_/nginx

docker run --name ngx -p xxxx:80 -d
-v {path local}:/usr/share/nginx/html:ro
nginx:1.18.0-alpine

- ***Portainer (Dashboard para Docker)***

vamos a crear un volumen para no perder la configuración de portainer

docker **volume** create **portainer_data**

Iniciamos el contenedor de Portainer:

docker **run** -d -p 9000:9000 -p 8000:8000 --name portainer --restart always
-v /var/run/docker.sock:/var/run/docker.sock
--mount type=bind,source=/var/run/docker.sock,target=/var/run/docker.sock
-v portainer_data:/data
--mount source=**portainer_data**,target=/data
portainer/portainer

https://portainer.readthedocs.io/en/stable/deployment.html

- ***Mysql Local***

docker run --name mysql -d

--restart always

-v xxx:/var/lib/mysql

-e MYSQL_ROOT_PASSWORD=root

-p xxxx:3306

mysql

Comando básicos

- docker run { --name -p -v -link}
- docker ps (https://docs.docker.com/engine/reference/commandline/ps/)
- docker stats
    - Cambiar Límites de memoria y Cpu: ejemplo: --cpus=".5" -m 200M
- docker logs -f --tail=10 <id>

Dockerizing una aplicación

Dockerfile

Como una mejor alternativa, podríamos recurrir al enfoque automatizado de crear las imágenes utilizando **Dockerfile**. Dockerfile es un script de compilación basado en texto que contiene instrucciones especiales en una secuencia para construir las imágenes correctas y relevantes a partir de las imágenes base. Las instrucciones secuenciales dentro del Dockerfile pueden incluir la selección de imagen base, instalar la aplicación requerida, agregar la configuración y los archivos de datos, y ejecutar automáticamente los servicios, así como exponer esos servicios al mundo externo. Por lo tanto, el sistema de construcción automatizado basado en Dockerfile ha simplificado el proceso de construcción de imágenes. También ofrece una gran flexibilidad en la forma en que se organizan las instrucciones de compilación y en la forma en que visualizan el proceso de compilación completo.

Dockerfile para aplicación Java Spring Boot

```
FROM openjdk:11-slim
LABEL maintainer="hmunoz@unrn.edu.ar"
LABEL "unrn.edu.ar"="UNRN"
LABEL "service"="API Rest"
ADD ./build/libs/*.jar app.jar
ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
EXPOSE 8080
```

FROM: Cómo se puede ver en el ejemplo, todo Dockerfile para crear una imagen con la cual vamos a poder crear contenedores, parte de una imagen base, en este caso para una aplicación String Boot solo necesitamos de openJdk. El primer paso es buscar una imagen base que tenga todo lo necesario para nuestro proyecto. Podemos encontrar en https://hub.docker.com/_/openjdk que esta disponible la imagen oficial de openJdk. Es siempre recomendable buscar las imágenes las livianas posibles, en este caso estamos usando una version **slim.**

LABEL: permite poner etiquetas en las imágenes, las cuales por medio de filtros no ayudan a buscar contenedores que usen estas imágenes. Ahí en Label en el que se puede especificar quien desarrolla la imagen con la clave maintainer.

ADD: con esta instrucción agrega el jar compilado dentro de la imagen que vamos a crear. Es todo lo que vamos a necesitar para poder ejecutar nuestra aplicación Spring Boot.

ENTRYPOINT: Nos permite definir el comando que se va a ejecutar cuando inicie el contenedor. Si necesitamos que siempre sea el mismo comando ENTRYPOINT es la mejor opción. Sintaxis (ENTRYPOINT ["executable", "param1", "param2"]). Para nuestro caso, vamos a ejecutar Java con el parámetro -jar y la aplicación app.jar. Cabe mencionar que se podria agregar mas parametros en la misma construcción de la imagen como por ejemplo *"-Dspring.profiles.active=prod"* para dejar definido el perfil de spring boot que queremos usar para esta imagen, pero este parámetro se puede definir después cuando ejecutamos docker run -e *"-Dspring.profiles.active=prod"* o también en el docker-compose.

EXPOSE: La instrucción EXPOSE informa a Docker que el contenedor escucha en los puertos de red especificados en tiempo de ejecución. Puede especificar si el puerto escucha en TCP o UDP, y el valor predeterminado es TCP si no se especifica el protocolo. EXPOSE no publica realmente el puerto. Funciona como un tipo de documentación entre la persona que construye la imagen y la persona que ejecuta el contenedor, sobre qué puertos están destinados a ser publicados. Para publicar realmente el puerto cuando se ejecuta el contenedor, use el indicador -p en la ejecución de Docker para publicar y asignar uno o más puertos, o el indicador -P para publicar todos los puertos expuestos y asignarlos a puertos de alto orden.

Más documentación en este link:
https://spring.io/guides/topicals/spring-boot-docker/

Crear la Imagen con el Dockerfile

Una vez definido el Dockerfile, podemos con el cliente de docker construir la imagne con la siguiente instrucción:

$: docker **build** -t <nombreImagen>:<version> ■

El parámetro -t hace referencia a un TAG y el parámetro siguiente puede ser cualquier cadena de texto. Por lo general se describe el nombre de la imagen y una versión. El punto

al final del comando ubica desde que path vamos a trabajar, por lo general, el proyecto fuente tiene el DockerFile en la raíz del proyecto.

Ejemplo: $: docker **build** -t curso-docker:1.1 ■

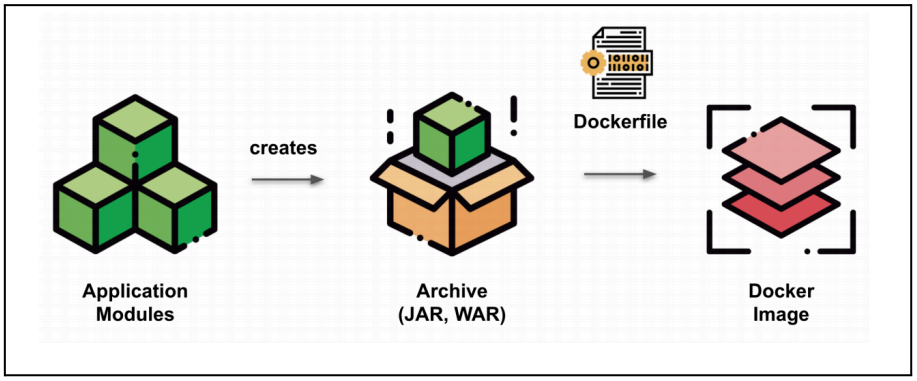

Fig 1. Crear Imagen Docker con un Dockerfile

En la figura 1 se puede ver la secuencias para obtener una imagen de Docker con un Dockerfile.

Dockerfile multistage

```
FROM node:lts-alpine as build-stage
WORKDIR /app
COPY *.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:1.17.9-alpine as production-stage
COPY frontend.conf /etc/nginx/nginx.conf
COPY --from=build-stage /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Es común también necesitar más de un paso para definir la construcción de una imagen de docker, esto se puede hacer con un Dockerfile multistage. En este ejemplo, vamos a crear un proyecto angular en un servidor nginx.

El primer paso, llamado *build-stage*, lo que permite es generar los javascripts, estilos y html con el código fuente de un proyecto angular. Para el caso de un proyecto angular necesitamos node, ejecutar npm install para recuperar las librerías que tenga el proyecto definido en el su definición y por último ejecutar npm run build, este comando crea el proyecto en una carpeta dist, para su distribución.

El segundo paso, llamado *production-stage*, toma esta carpeta dist y la copia dentro de una nueva imagen base de un servidor nginx.

Operaciones con la Imagen

Una vez definido nuestro Dockerfile podemos crear una imagen de docker, con la cual vamos a poder crear contenedores que vana brindar el servicios que necesitemos.
Como vimos en el apartado anterior, el comando para crear una imagen tiene la siguiente forma: $: docker **build -t** <nombreImagen>:<version> **.**

Una vez creada la imagen podemos verificar mediante el siguiente comando que está presente de forma local: $: docker **image** ls.

Esta imagen podríamos necesitar publicarla a un Registro de Docker privado o público. En caso de necesitar publicar la imagen en un registro publico debemos realizar las siguientes operaciones:

docker build -t 127.0.0.1:5000/backend:1.0 . (crea la imagen)

docker push 127.0.0.1:5000/backend:1.0 (publica la imagen)

127.0.0.1:5000 es un registro privado que esta como servicio en la máquina local, este servicio podría estar en otro dirección.

Docker Compose Ej: BackEnd Java + MongoDB

Compose es una herramienta para definir y ejecutar aplicaciones Docker de contenedores múltiples. Con Compose, utiliza un archivo YAML para configurar los servicios de su aplicación. Luego, con un solo comando, crea e inicia todos los servicios desde su configuración.

Supongamos que necesitamos ejecutar el servicio backend que definimos anteriormente con una base mongoDB. Este proceso lo podríamos hacer con comandos de docker con la siguiente secuencia:

Primero creamos un volumen para guardar la base de datos

$: docker volume create mongodb-curso

Después iniciamos en contenedor de mongo con el volumen, con la referencia a la carpeta donde mongo guarda los datos y exponiendo el puerto 27027 que utiliza mongo por defecto al mismo puerto en nuestro host, en caso que necesitemos acceder con un cliente.

$: docker run -d --name curso-mongo
 -p 27017:27017 (solo si queremos acceder de forma local a la base)
-v mongodb-curso:/data/db
mongo:4.2.6

Para finalizar, ejecutamos el contenedor del backend con la referencia a la base mongo utilizando el tag de links.

$: docker run --name curso-backend
-p 9010:8080 (vamos a poder acceder al servicio en el puerto 9010)
--link curso-mongo:mongodb (Vinculación con la base de datos por referencia)
backend:1.0

Este proceso se puede reemplazar con un solo archivo de configuración, generalmente llamado docker-compose.YML. Para lograr el mismo resultado se definen dos servicios, uno para una base de datos Mongo y otro para un servicio de backend en java, el mismo que definimos anteriormente con el Dockerfile basado en una imagen openjdk:11-slim. De esta forma se define:

```
version: '3.3'
services:
 curso-mongo:
   image: mongo:4.2.6
```

```
  volumes:
   - mongodb-cursos:/data/db
  ports:
    - '27017:27017'
 curso-backend:
  image: backend:1.0
  environment:
    - SPRING_PROFILES_ACTIVE=docker
  ports:
    - '9010:8080'
  depends_on:
    - curso-mongo
volumes:
 mongodb-cursos:
```

Con el siguiente comando podemos iniciar el servicio con todos los contenedores que estén definidos:

- docker-compose -d up (-d desechado)
- docker-compose -f docker-compose.yml -up (-f debemos indicar el yml)

Para eliminar los contenedores utilizamos el siguiente comando:

- docker-compose down

También podemos ejecutar docker-compose ps para ver los contenedores que tiene docker compose

```
         Name                        Command               State            Ports
-------------------------------------------------------------------------------------------
backend_curso-backend_1   java -Djava.security.egd=f ...   Up      0.0.0.0:9010->8080/tcp
backend_curso-mongo_1     docker-entrypoint.sh mongod      Up      0.0.0.0:27017->27017/tcp
```

Bibliografía

- https://docs.docker.com/get-started/overview/
- https://www.docker.com/resources/what-container
- https://geekflare.com/docker-architecture/
- Learning Docker ISBN 978-1-78439-793-7
    - https://drive.google.com/file/d/1MelM5LeRF6GuHVe15pyn7irr0UQzyCLM/view?usp=sharing
- Hands-On Microservices with Spring Boot and Spring Cloud
    - ISBN 978-1-78961-347-6
    - https://drive.google.com/file/d/1Qx7hMyxdVOtQUuYC48VYErss5Z5iNv-1/view?usp=sharing
- https://spring.io/guides/topicals/spring-boot-docker/